Lab 1

1. **`Date` Class Modification**

Modify the `Date` class in Problem 1 of Lab 1. The new version should have the following overloaded operators:

- `++` ***Prefix and postfix increment operators.*** These operators should increment the object's day member.
- `--` ***Prefix and postfix decrement operators.*** These operators should decrement the object's day member.
- `-` Subtraction operator. If one `Date` object is subtracted from another, the operator should give the number of days between the two dates. For example, if April 10, 2014 is subtracted from April 18, 2014, the result will be 8.
- `<<` **`cout`*'s stream insertion operator.*** This operator should cause the date to be displayed in the form April 18, 2014
- `>>` **`cin`*'s stream extraction operator.*** This operator should prompt the user for a date to be stored in a Date object.

The class should detect the following conditions and handle them accordingly:

- When a date is set to the last day of the month and incremented, it should become the first day of the following month.
- When a date is set to December 31 and incremented, it should become January 1 of the following year.
- When a day is set to the first day of the month and decremented, it should become the last day of the previous month.
- When a date is set to January 1 and decremented, it should become December 31 of the previous year.

Demonstrate the class's capabilities in a simple program.

Input Validation: The overloaded >> operator should not accept invalid dates. For example, the date 13/45/2014 should not be accepted.

2. Corporate Sales

A corporation has six divisions, each responsible for sales to different geographic locations. Design a `DivSales` class that keeps sales data for a division, with the following members:

- An array with four elements for holding four quarters of sales figures for the division
- A private static variable for holding the total corporate sales for all divisions for the entire year
- A member function that takes four arguments, each assumed to be the sales for a quarter. The value of the arguments should be copied into the array that holds the sales data. The total of the four arguments should be added to the static variable that holds the total yearly corporate sales

- A function that takes an integer argument within the range of 0–3. The argument is to be used as a subscript into the division quarterly sales array. The function should return the value of the array element with that subscript

Write a program that creates an array of six `DivSales` objects. The program should ask the user to enter the sales for four quarters for each division. After the data are entered, the program should display a table showing the division sales for each quarter. The program should then display the total corporate sales for the year.

Input Validation: Only accept positive values for quarterly sales figures.

3. `ShiftSupervisor` Class

In a particular factory a shift supervisor is a salaried employee who supervises a shift. In addition to a salary, the shift supervisor earns a yearly bonus when his or her shift meets production goals. Design a `ShiftSupervisor` class that is derived from the `Employee` class you created in Problem 1 in Assignment 1. The `ShiftSupervisor` class should have a member variable that holds the annual salary and a member variable that holds the annual production bonus that a shift supervisor has earned. Write one or more constructors and the appropriate accessor and mutator functions for the class. Demonstrate the class by writing a program that uses a `ShiftSupervisor` object.

4. `Essay` Class

Design an `Essay` class that is derived from the `GradedActivity` class presented below. The `Essay` class should determine the grade a student receives on an essay. The student's essay score can be up to 100, and is determined in the following manner:

- Grammar: 30 points
- Spelling: 20 points
- Correct length: 20 points
- Content: 30 points

Demonstrate the class in a simple program.

Contents of `GradeActivity.h`

```
#ifndef GRADEDACTIVITY_H
#define GRADEDACTIVITY_H

// GradedActivity class declaration

class GradedActivity
{
      private:
      double score; // To hold the numeric score
      public:
            // Default constructor
            GradedActivity()
                  { score = 0.0; }
```

```
            // Constructor
            GradedActivity(double s)
                  { score = s; }

            // Mutator function
            void setScore(double s)
                  { score = s; }

            // Accessor functions
            double getScore() const
                  { return score; }

            char getLetterGrade() const;
};
#endif
```

Contents of `GradedActivity.cpp`

```
#include "GradedActivity.h"

//*****************************************************
// Member function GradedActivity::getLetterGrade *
//*****************************************************

char GradedActivity::getLetterGrade() const
{
      char letterGrade; // To hold the letter grade

      if (score > 89)
            letterGrade = 'A';
      else if (score > 79)
            letterGrade = 'B';
      else if (score > 69)
            letterGrade = 'C';
      else if (score > 59)
            letterGrade = 'D';
      else
            letterGrade = 'F';

      return letterGrade;
}
```

5. Course Grades

In a course, a teacher gives the following tests and assignments:

- A **lab activity** that is observed by the teacher and assigned a numeric score
- A **pass/fail exam** that has 10 questions. The minimum passing score is 70
- An **essay** that is assigned a numeric score
- A **final exam** that has 50 questions

Write a class named `CourseGrades`. The class should have a member named `grades` that is an array of `GradedActivity` pointers. The `grades` array should have four elements, one for each of the assignments previously described. The class should have the following member functions:

`setLab`: This function should accept the address of a `GradedActivity` object as its argument. This object should already hold the student's score for the lab activity. Element 0 of the `grades` array should reference this object.

`setPassFailExam`: This function should accept the address of a `PassFailExam` object as its argument. This object should already hold the student's score for the pass/fail exam. Element 1 of the `grades` array should reference this object.

`setEssay`: This function should accept the address of an `Essay` object as its argument. (See Problem 4 for the `Essay` class. If you have not completed Problem 4, use a `GradedActivity` object instead.) This object should already hold the student's score for the essay. Element 2 of the `grades` array should reference this object.

`setPassFailExam`: This function should accept the address of a `FinalExam` object as its argument (showed below). This object should already hold the student's score for the final exam. Element 3 of the grades array should reference this object.

`print`: This function should display the numeric scores and grades for each element in the `grades` array.

Demonstrate the class in a program.

Contents of `FinalExam.h`

```
#ifndef FINALEXAM_H
#define FINALEXAM_H
#include "GradedActivity.h"

class FinalExam : public GradedActivity
{
      private:
            int numQuestions; // Number of questions
            double pointsEach; // Points for each question
            int numMissed; // Number of questions missed
      public:
            // Default constructor
            FinalExam()
                  {     numQuestions = 0;
                        pointsEach = 0.0;
                        numMissed = 0; }
```

```
            // Constructor
            FinalExam(int questions, int missed)
                  { set(questions, missed); }

            // Mutator function
            void set(int, int); // Defined in FinalExam.cpp

            // Accessor functions
            double getNumQuestions() const
                  { return numQuestions; }

            double getPointsEach() const
                  { return pointsEach; }
            int getNumMissed() const
                  { return numMissed; }
};
#endif
```

Contents of `FinalExam.cpp`

```
#include "FinalExam.h"

//*******************************************************
// set function *
// The parameters are the number of questions and the *
// number of questions missed. *
//*******************************************************

void FinalExam::set(int questions, int missed)
{
      double numericScore; // To hold the numeric score

      // Set the number of questions and number missed.
      numQuestions = questions;
      numMissed = missed;

      // Calculate the points for each question.
      pointsEach = 100.0 / numQuestions;

      // Calculate the numeric score for this exam.
      numericScore = 100.0 - (missed * pointsEach);

      // Call the inherited setScore function to set
      // the numeric score.
      setScore(numericScore);
}
```

6. File Filter

A file filter reads an input file, transforms it in some way, and writes the results to an output file. Write an abstract file filter class that defines a pure virtual function for transforming a character. Create one derived class of your file filter class that performs encryption, another that transforms a file to all uppercase, and another that creates an unchanged copy of the original file. The class should have the following member function:

```
void doFilter(ifstream &in, ofstream &out)
```

This function should be called to perform the actual filtering. The member function for transforming a single character should have the prototype:

```
char transform(char ch)
```

The encryption class should have a constructor that takes an integer as an argument and uses it as the encryption key.

7. Date Exceptions

Modify the `Date` class you wrote for Problem 1 of Assignment 1. The class should implement the following exception classes:

`InvalidDay`	Throw when an invalid day (< 1 or > 31) is passed to the class.
`InvalidMonth`	Throw when an invalid month (< 1 or > 12) is passed to the class.

Demonstrate the class in a driver program.

8. SortableVector Class Template

Write a class template named `SortableVector`. The class should be derived from the `SimpleVector` class presented in Assignment 4. It should have a member function that sorts the array elements in ascending order. (Use the sorting algorithm of your choice.) Test the template in a driver program.

9. IntArray Class Exception

An IntArray class dynamically creates an array of integers and performs bounds checking on the array. If an invalid subscript is used with the class, it displays an error message and aborts the program. Modify the class so it throws an exception instead.

Contents of `IntArray.h`

```
// Specification file for the IntArray class
#ifndef INTARRAY_H
#define INTARRAY_H

class IntArray
{
      private:
            int *aptr; // Pointer to the array
            int arraySize; // Holds the array size
```

```
            void subscriptError(); // Handles invalid subscripts
      public:
            IntArray(int); // Constructor
            IntArray(const IntArray &); // Copy constructor
            ~IntArray(); // Destructor

            int size() const // Returns the array size
                  { return arraySize;}

            int &operator[](const int &); // Overloaded [] operator
};
#endif
```

Contents of IntArray.cpp

```
// Implementation file for the IntArray class
#include <iostream>
#include <cstdlib> // For the exit function
#include "IntArray.h"
using namespace std;

//*******************************************************
// Constructor for IntArray class. Sets the size of the *
// array and allocates memory for it. *
//*******************************************************

IntArray::IntArray(int s)
{
      arraySize = s;
      aptr = new int [s];
      for (int count = 0; count < arraySize; count++)
            *(aptr + count) = 0;
}

//*****************************************************
// Copy Constructor for IntArray class. *
//*****************************************************

IntArray::IntArray(const IntArray &obj)
{
      arraySize = obj.arraySize;
      aptr = new int [arraySize];
      for(int count = 0; count < arraySize; count++)
            *(aptr + count) = *(obj.aptr + count);
}

//*****************************************************
```

```
// Destructor for IntArray class. *
//*****************************************************

IntArray::~IntArray()
{
      if (arraySize > 0)
            delete [] aptr;
}

//*************************************************************
// subscriptError function. Displays an error message and *
// terminates the program when a subscript is out of range. *
//*************************************************************

void IntArray::subscriptError()
{
      cout << "ERROR: Subscript out of range.\n";
      exit(0);
}
//*****************************************************
// Overloaded [] operator. The argument is a subscript. *
// This function returns a reference to the element *
// in the array indexed by the subscript. *
//*****************************************************

int &IntArray::operator[](const int &sub)
{
      if (sub < 0 || sub >= arraySize)
      subscriptError();
      return aptr[sub];
}
```

class